

User Story seleccionada

Para este trabajo práctico se seleccionó la User Story **“Inscribirme a actividad”**, correspondiente al sistema **Ecoharmony Park**. La funcionalidad permite que un usuario pueda inscribirse a una actividad del parque, seleccionando actividad, horario, cantidad de participantes y completando los datos requeridos.

Aplicación de TDD

La funcionalidad se desarrolló aplicando el enfoque **TDD**, siguiendo el ciclo **Red-Green-Refactor**.

Primero se escribieron pruebas automatizadas con **Vitest** para validar el comportamiento esperado. (RED)

Luego se implementó el código mínimo necesario para que las pruebas pasaran. (GREEN)

Finalmente, se realizaron mejoras en la organización del código sin modificar el resultado esperado. (REFACTOR)

Las pruebas cubren validaciones como horario permitido del parque, días en los que el parque está cerrado y días festivos, disponibilidad de horarios, cupos disponibles, cantidad válida, datos de participantes y envío del email de confirmación.

Decisiones de diseño tomadas

Se decidió separar la lógica en funciones específicas para que el código sea más claro y fácil de probar.

Por ejemplo, se crearon funciones como `validarHorarioParque`, `validarDiaParqueAbierto`, `validarCantidadDisponible`, `validarNombre`, `validarDNI` y `validarEdad`.

También se separó la lógica relacionada al email en funciones propias, como `construirHTMLEmail` y `enviarEmailInscripcion`. Esto permite que el armado del correo y el envío puedan probarse de forma independiente.

Además, se utilizó un archivo JSON como base de datos simulada, donde se guardan las actividades, usuarios, horarios y cupos disponibles.

Esta decisión simplifica el desarrollo del trabajo ya que nos permite probar funcionalidades sin depender de una base de datos real.

Convenciones de código utilizadas

Nombres de variables y funciones: usamos **camelCase** para variables, funciones y métodos, Por ejemplo:

- `getActividades`
- `getUsuario`
- `validarHorarioParque`
- `validarCantidadDisponible`
- `mensajeCupoSuperado`

Esta decisión permite que los nombres sean descriptivos y fáciles de entender dentro del código.

Constantes

Para las **constantes** se utilizó **SNAKE_CASE**, es decir, palabras separadas por guiones bajos. Además cada palabra será escrita en mayúscula, Por ejemplo:

- HORA_PRIMER_TURN0
- HORA_ULTIMO_TURN0

Esto permite identificar rápidamente que son valores constantes y que no deberían modificarse durante la ejecución del programa.

Formato automático

Se utilizó **Prettier** para formatear el código automáticamente. Esto permite mantener una estructura consistente en todos los archivos del proyecto, evitando diferencias de formato entre integrantes del grupo.

Prettier ayuda a ordenar:

- sangrías
- saltos de línea
- espacios
- comas
- llaves
- longitud visual de las líneas

Sangría e indentación

Se utilizó una sangría uniforme de 2 espacios, siguiendo el estilo común en proyectos JavaScript y el formato aplicado por Prettier.

Ejemplo:

```
if (!actividad) {  
    return null;  
}
```

Espacios

Se dejaron espacios alrededor de operadores y después de comas para mejorar la lectura del código. Este criterio también es aplicado automáticamente por Prettier.

Ejemplo:

```
return cantidad <= cupos;
```

También se usaron espacios entre parámetros cuando una función recibe más de un dato:

```
validarCantidadDisponible(actividadId, horario, cantidad);
```

Llaves y bloques de código

Las llaves se colocaron en la misma línea de la función, condición o bloque correspondiente. Este formato coincide con el estilo aplicado por Prettier.

Ejemplo:

```
export function getActividad(id) {  
    return db.actividades.find((a) => a.id === id) ?? null;  
}
```

Longitud de línea

Se utilizó Prettier para organizar automáticamente los saltos de línea cuando una instrucción es demasiado larga.

Prettier utiliza por defecto un `printWidth` de **80 caracteres por línea**

Organización del código

El código se organizó separando responsabilidades:

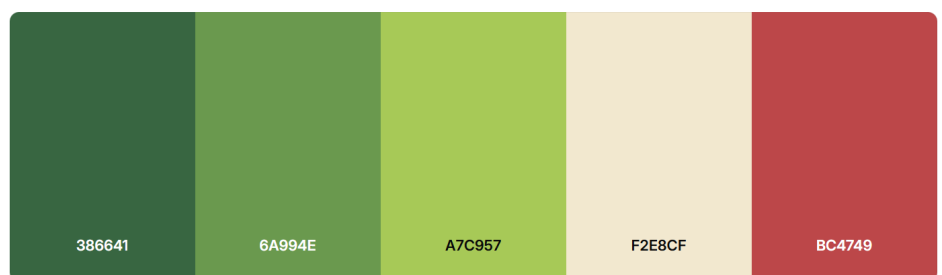
- `index.js`: funciones principales, validaciones y acceso a datos.
- `app.js`: lógica de la interfaz y manejo del formulario.
- `email.js`: construcción y envío del email de confirmación.
- archivos de test: pruebas automatizadas con Vitest.
- `style.css`: estilos visuales y paleta de colores.

Esta separación permite que cada archivo tenga una responsabilidad clara y que las funciones puedan probarse de manera más simple.

Diseño visual

En cuanto al diseño visual se utilizó la paleta de colores indicada por el PO: **“Fresh Greens”**

Estos colores ayudan a mantener una identidad visual ecológica.



Conclusión

Las decisiones tomadas permitieron construir funcionalidades simples, claras y testeables.

El uso de TDD ayudó a verificar cada comportamiento antes de avanzar con la implementación.

Además, las convenciones de código, el uso de Prettier y la separación de responsabilidades mejoran la legibilidad y facilitan futuras modificaciones del sistema.